

Contents

Chess Next-Move Prediction with N-Gram, LSTM, and Hybrid CNN+LSTM Models	1
Abstract	1
1. Problem definition	1
2. Data preparation	2
3. Models	4
4. Experimental setup	6
5. Results	7
6. Performance analysis	8
7. Limitations and future work	9
8. Conclusion	10
References	10

Chess Next-Move Prediction with N-Gram, LSTM, and Hybrid CNN+LSTM Models

Abstract

We frame chess next-move prediction as a sequence-modelling task on human play and compare three models trained on 300 000 filtered games from the Lichess Open Database (January 2017): a trigram baseline with Katz back-off, a 2-layer LSTM (5.17 M parameters) over UCI move sequences, and a hybrid CNN+LSTM (8.0 M parameters) that adds an $8 \times 8 \times 18$ board-tensor branch to the LSTM. On a held-out 15 000-game test split (1.16 M positions), the hybrid achieves top-1 accuracy 0.408 and perplexity 8.03, versus 0.371 / 10.51 for the LSTM and 0.252 / 156.7 for the baseline. The most informative finding is the phase-by-phase shape: adding board awareness (LSTM $\rightarrow$ hybrid) yields essentially zero gain in the opening but improves perplexity by 47 % in the endgame — board state matters most precisely where pure-sequence context becomes least informative. All three models support legal-move masking at inference, so they cannot recommend illegal or invented moves.

1. Problem definition

Given a chess game’s history of moves, predict the next move played. Concretely: at every position before move i , we model $P(\text{move}_i \mid \text{move}_0, \dots, \text{move}_{\{i-1\}})$ (and, for the hybrid model, additionally on the board state at that position). This is a *behaviour-prediction* problem — we predict what humans typically play, not what is theoretically optimal. A model approximating optimal play would be a different problem, one already well-served by chess engines such as Stockfish.

The task is sequential because a chess game is a discrete sequence of moves where each move’s distribution depends on the cumulative game state: opening choices constrain middlegame plans, plans shape piece placement, piece placement determines tactical opportunities. We tokenise every move using UCI (Universal Chess Interface) notation — fixed-width strings such as e2e4

(the piece on e2 moves to e4), g8f6, or e7e8q (pawn on e7 promotes to queen on e8). UCI gives a closed vocabulary that covers every legal move in every legal position, with no ambiguity from positional context.

Throughout this report we use **plies** — a chess term meaning a single half-move. One full move (e.g. 1. e4 e5) consists of two plies. A typical Rapid/Classical game in our corpus runs 80–120 plies (40–60 full moves).

We evaluate every model with three metrics:

- **Top-1 / Top-3 / Top-5 accuracy:** fraction of test positions where the move actually played is among the model’s K highest-probability predictions, restricted to the legal moves available in the current position.
- **Perplexity:** $\exp(\text{mean cross-entropy})$ over the predicted-move distribution. Lower is better; intuitively, perplexity p means the model is as uncertain as if it had to guess uniformly between p candidate moves.
- **Per-phase breakdown:** opening (plies 0–19), early middlegame (20–39), late middlegame (40–79), endgame (80+).

The motivating use case for this kind of model is automated chess teaching: tools that play optimally are abundant, but tools that predict *realistic human moves at a target playing strength* have far fewer instances (Maia Chess [1] is the closest prior art). This report’s models lay technical groundwork for the prediction component of such a pipeline.

2. Data preparation

2.1 Source and filtering

Source: Lichess Open Database monthly dump for 2017-01 ($\approx$ 1.9 GB compressed, $\approx$ 5 M raw games). We apply project-specific filters that retain only games likely to contain coherent, skilled play:

- `TimeControl` $\geq$ 480 estimated seconds (Rapid + Classical only — bullet/blitz are noisy and time-pressured)
- Both `WhiteElo` $\geq$ 1500 and `BlackElo` $\geq$ 1500 (skill floor; below this, blunder noise dominates the move distribution)
- `Termination` == "Normal" (excludes timeouts, disconnects, abandons)
- $40 \leq \text{plies} \leq 240$ (excludes near-instant resignations and outlier-length games)

These filters are applied in our parsing pipeline; the Lichess database does not expose them as download parameters. After filtering we retain the first 300 000 games as our corpus.

2.2 Streaming parser with header-only fast-rejection

Each PGN game has two parts: a *header* (metadata block with `TimeControl`, `Elo`, `Termination`, etc.) and a *body* (the move list). Parsing the body is the expensive step because each move string must be converted into an internal `Move` object. A naive parser that fully decodes every game’s body before testing the header filters spent $\sim$ 60 minutes slicing 300 000 games from 2017-01.

We resolve this by adding a custom python-chess *visitor* (`_HeaderFilterVisitor` in `src/data/pgn_parser.py`). The visitor monitors the parser and, the moment the header has been read but before the body is decoded, checks whether the header passes our filters. If it doesn't, the visitor returns a SKIP signal that tells the parser to advance to the next game without decoding the body. Since ~80 % of Lichess games are bullet/blitz that fail our TimeControl filter, this skips the slow step for ~80 % of games and reduced our slicing time to ~15 minutes. The set of games kept is identical (verified by the existing five unit tests on filter behaviour); only the work spent on rejected games is eliminated.

2.3 Tokenisation and vocabulary

The model treats each game as a sequence of “words” where every word is a chess move in UCI notation. Predicting the next move is therefore the chess analogue of predicting the next word in a sentence — the same setup as a language model, but with chess moves as the vocabulary. The vocabulary contains four reserved special tokens (`<PAD>` = 0, `<START>` = 1, `<END>` = 2, `<UNK>` = 3) followed by the sorted set of UCI moves observed in the training split, built deterministically with `seed = 42`. For our 255 000-game train split this produces 1 940 tokens; the theoretical maximum is approximately 1 972 (every legal UCI move on a chessboard).

The four special tokens serve distinct purposes: `<PAD>` is padding filler used when training batches contain games of different lengths, so they all line up to the longest in the batch (the loss function ignores PAD positions during training); `<START>` is a start-of-game marker prepended to every move sequence so the model has something to condition on when predicting the first move; `<END>` is an end-of-game marker appended so the model can learn to predict that the game has terminated rather than always producing some plausible-looking continuation; `<UNK>` is a fallback for any UCI string not seen in training (almost never used in practice but provides a safety net so the model never crashes on unfamiliar input). All four are excluded from the inference-time top-K candidate set, so the model can never recommend a special token as if it were a move.

2.4 Board encoding

Each position is encoded as an $8 \times 8 \times 18$ float32 tensor:

- **12 piece planes** — one per piece type and colour: white Pawn (P), Knight (N — using “N” because “K” is taken by the King), Bishop (B), Rook (R), Queen (Q), King (K), then the same six for black. Each plane has a 1.0 at every square occupied by that piece type and 0.0 elsewhere.
- **1 turn plane** — uniformly 1.0 if it is white to move, else 0.0.
- **4 castling-rights planes** — white kingside, white queenside, black kingside, black queenside. Each is a uniform plane of 1.0 if the corresponding right is still available, else 0.0. Castling rights matter as a behaviour signal: a player with kingside castling rights is preparing to castle; once the rights are lost the player's plan changes structurally.
- **1 en-passant target plane** — a single 1.0 at the en-passant target square if any, else all zeros.

We deliberately exclude tactical channels (attack maps, pins, hanging pieces) so that any tactical understanding the model exhibits is genuinely learned from raw piece positions rather than precomputed and handed in.

2.5 Splits

85 / 10 / 5 train / val / test, partitioned *by game* (never by position) with `random.Random(seed=42).shuffle`. Splitting by position would leak entire openings between train and test. Sizes: 255 000 train, 30 000 val, 15 000 test games — yielding ≈ 19.78 M, 2.33 M, and 1.16 M individual position-prediction samples respectively.

2.6 Two dataset granularities

The LSTM trains on whole games (one game per sample, variable length); the hybrid trains on individual positions (one position per sample, each carrying a board snapshot, the move history up to that point, and the move that was actually played next). The hybrid needs a *board picture* per sample, which only makes sense at one specific moment in time, hence the per-position view; the LSTM has no such constraint and benefits from seeing entire games at once.

3. Models

3.1 N-gram baseline

`src/models/ngram.py` — TrigramKatz. Pure Python with no machine-learning framework. We count trigrams (a, b, c) over the training games (with two leading <START> tokens and a trailing <END> token per game), and at inference compute $P(c \mid a, b)$ via Katz back-off with absolute discount $d = 0.5$:

- If trigram (a, b, c) was seen count times in context (a, b) of total mass T, then $P(c \mid a, b) = (\text{count} - d) / T$.
- Otherwise back off to bigram $P(c \mid b)$, weighted by the discount-reserved mass for context (a, b).
- Bigram falls back analogously to a Laplace-smoothed unigram so no probability is ever zero.

$d = 0.5$ is the standard textbook default for absolute discount; we did not tune it. Counts are stored as plain dicts (the model pickles cleanly), and per-context back-off weights are cached lazily on first use. The fitted model holds 1 870 unigram entries, 175 144 bigram contexts, and several million trigram contexts; the saved gzipped pickle is 37 MB.

3.2 LSTM

`src/models/lstm.py` — MoveLSTM. The architecture is a classic three-stage neural sequence model:

- An **Embedding** layer maps each move ID (0–1939) into a 256-dimensional dense vector.
- A **2-layer LSTM** with hidden size 512 and 0.2 dropout reads the sequence of move embeddings left-to-right, maintaining a running hidden state that summarises the game so far.
- A **linear output layer** projects the LSTM’s hidden state at each timestep into a 1 940-dimensional logit vector — one logit per possible next move.

At training time, position t of the input sequence is asked to predict the move at position $t+1$ (standard left-to-right next-token prediction). At inference, we take the LSTM’s hidden state after the last played move and use the logits at that timestep to produce a probability distribution over the next move. Total parameters: 5 170 068.

The choice of 2 layers, hidden 512, embedding 256 is the standard “small but real” sequence-modelling configuration — large enough to learn the ~1 940-token UCI distribution over typical game lengths, small enough to train within a free-tier GPU session. We did not pursue transformer architectures because the focus of the project is on whether the sequence model improves on the trigram baseline rather than on architectural novelty.

3.3 Hybrid CNN + LSTM

`src/models/hybrid.py` — `MoveBoardHybrid`. The hybrid extends the LSTM with a second input branch that consumes the $8 \times 8 \times 18$ board tensor through a 3-layer convolutional neural network (CNN). At every prediction, the move-history branch produces a 512-dimensional summary and the board branch produces a 256-dimensional summary; the two summaries are concatenated into a single 768-dimensional vector and passed through a final linear layer that emits the next-move logits over the 1 940-token vocabulary.

The CNN branch consists of three same-padded 3×3 convolutional layers (channel sizes $18 \rightarrow 64 \rightarrow 128 \rightarrow 128$, ReLU activation between them) followed by a flatten and a linear projection from the 8 192-element flattened activation down to the 256-dimensional board feature. Same-padding keeps the spatial dimension at 8×8 throughout the stack so the model retains full per-square activations until flattening. Total parameters: 7 995 988.

The intuition behind the architecture is that the LSTM branch captures “what kind of game this is” (opening structure, plan progression, recent trades), while the CNN branch captures “what the position looks like right now” (piece relationships, threatened structures, endgame patterns). Whichever signal is more informative for the current position dominates the final prediction through the learned weights of the shared head.

3.4 Legal-move masking

All three models share an identical inference contract: `predict_topk(history, k, legal_token_ids)` returns the top-K candidates from the legal-move set with `<PAD>`, `<START>`, and `<END>` always excluded. The model itself can assign internal probability mass to illegal moves — it is not given any built-in chess-rule constraint — but those moves are filtered before reaching the user. The demo and the evaluation harness can therefore never recommend an illegal or invented move; the worst possible failure is a strategically poor *legal* move.

4. Experimental setup

4.1 Training configurations

Hyperparameter	LSTM	Hybrid
Optimiser	AdamW	AdamW
Learning rate	1×10^{-3}	1×10^{-3}
Weight decay	0.0	0.0
Gradient clip (L2)	1.0	1.0
LR scheduler	ReduceLROnPlateau (val loss, factor 0.5, patience 1)	same
Batch size	64 games per step (LSTM trains on whole games)	256 positions per step (hybrid trains on individual positions)
Epochs	8	4
Loss	CrossEntropy, ignore_index = <PAD>	CrossEntropy
Best-checkpoint criterion	Lowest val loss	Lowest val loss

An *epoch* is one complete pass through the training data. The hybrid’s 4 epochs are sufficient because each hybrid epoch sees roughly 80× more individual training samples than each LSTM epoch — each game contributes one sample to the LSTM but ~80 samples (one per ply) to the hybrid. Hybrid 4 epochs is therefore comparable to LSTM ~320 epochs in terms of total samples seen during optimisation. Both models converge well within these budgets; both validation curves were still improving at their respective cut-offs.

4.2 Evaluation harness

`src/training/evaluate.py` defines a uniform `Predictor` interface with a single `score_game(moves, k)` method that returns per-ply position scores. Each model has a thin adapter (`NgramPredictor`, `LSTMPredictor`, `HybridPredictor`); the harness then runs aggregated top-K accuracy, perplexity, and phase-bucketed metrics through the same code path for all three models, so comparisons are mechanically apples-to-apples on the same vocabulary and the same test split.

4.3 Reproducibility

All randomness flows from `seed = 42`: the data splits, the vocabulary order, the per-epoch training shuffling. Saved checkpoints include their architectural configuration so they reload into a fresh `nn.Module` without external metadata. The codebase has 82 passing unit tests covering the data pipeline, models, and evaluation harness. Hardware does not affect model output; identical code, data, and seed reproduce identical models within floating-point noise.

5. Results

5.1 Three-way comparison on the test split

All three models evaluated on the same 1 162 103 position-prediction pairs from the same 15 000-game test split, using the same 1 940-token vocabulary.

Metric	n-gram	LSTM	Hybrid	LSTM vs n-gram	Hybrid vs LSTM
Top-1	0.2519	0.3707	0.4076	+11.9 pp	+3.7 pp
Top-3	0.4576	0.6301	0.6748	+17.3 pp	+4.5 pp
Top-5	0.5626	0.7438	0.7840	+18.1 pp	+4.0 pp
Perplexity	156.70	10.51	8.03	14.9× lower	24 % lower

5.2 Per-phase breakdown (top-1 accuracy and perplexity)

Phase (plies)	n posi- tions	n-gram	LSTM	Hybrid	n-gram pp	LSTM pp	Hybrid pp
Open- ing (0–19)	300 000	0.366	0.474	0.473	18.0	5.37	5.34
Early- mid (20– 39)	300 000	0.213	0.369	0.393	163	9.81	8.56
Late- mid (40– 79)	401 899	0.187	0.303	0.361	540	16.4	10.8
Endgame (80+)	160 204	0.275	0.352	0.431	373	13.8	7.35

5.3 Generalisation cross-checks

- LSTM: validation perplexity 10.62, test perplexity 10.51 — difference 0.11.
- Hybrid: validation perplexity 8.04, test perplexity 8.03 — difference 0.01.

Both models are calibrated to the held-out distribution with no overfitting signal; the small val/test gaps are within noise of one epoch’s improvement.

5.4 Training trajectories

LSTM validation perplexity decreased monotonically over 8 epochs: 22.48 → 15.89 → 13.53 → 12.36 → 11.68 → 11.19 → 10.86 → 10.62. The validation loss curve was still falling at epoch 8, suggesting a longer run could yield approximately 0.2–0.5 perplexity more.

Hybrid validation perplexity over 4 epochs: $9.56 \rightarrow 8.62 \rightarrow 8.24 \rightarrow 8.04$. Same observation: still improving at the cut-off, but with clear diminishing returns.

6. Performance analysis

The two cleanest findings emerge from the per-phase breakdown.

6.1 The LSTM closes the n-gram’s middlegame gap

The n-gram peaks in the opening (top-1 = 0.366) — exactly where its memorisation strength matters: with millions of trigram contexts seen across 255 000 training games, common opening continuations are well-covered, and for unseen contexts the bigram and Laplace unigram fallbacks remain reasonable. Where the n-gram fails is the middlegame: top-1 collapses to 0.187 in late-middlegame because moves 21–40 are where positions diverge into the long tail of unique configurations, and a three-token context window cannot encode the structural information needed.

The LSTM closes that gap. Its biggest absolute gain over the baseline appears in early-middlegame (+0.156 top-1) and late-middlegame (+0.116 top-1) — the two phases where the n-gram bottoms out. The full-game history compressed into the LSTM’s hidden state encodes opening choice, plan execution, and trade dynamics in a way the trigram window cannot.

6.2 The hybrid’s gain over the LSTM grows monotonically through the game

This is the more interesting finding because it is *predicted by first principles*: board awareness should add the most value precisely where pure-sequence context becomes least informative. The data confirms this exactly:

Phase	Hybrid top-1 Δ vs LSTM	Hybrid perplexity improvement over LSTM
Opening	−0.001 (tied)	tied (5.34 vs 5.37)
Early-middlegame	+0.024	13 % (8.56 vs 9.81)
Late-middlegame	+0.058	34 % (10.8 vs 16.4)
Endgame	+0.079	47 % (7.35 vs 13.8)

In the opening, the LSTM and the hybrid are *tied*. Adding the $8 \times 8 \times 18$ board tensor and a 3-layer CNN contributes essentially nothing to opening predictions, because openings are *defined* by their move sequences, and any board state reachable from a given opening sequence is implied by it. The hybrid pays for its extra parameters here and gains nothing in return.

By endgame the picture is opposite. Move history matters less (most of those moves were trades that have already resolved), and the surviving-piece configuration becomes the dominant signal. Hybrid top-1 of 0.431 versus LSTM 0.352 is a +7.9 pp absolute gain; perplexity 7.35 versus 13.8 is a 47 % improvement. The CNN branch over the board has earned its place.

This is methodologically the right kind of finding because it falsifies the alternative hypothesis (“just adding more parameters is what helped”). If raw parameter count were doing the work, the gain should have been roughly uniform across phases. The phase-shape monotonicity is direct evidence that the *board signal*, not added capacity, is what improves predictions in the late game.

6.3 Qualitative observations

In interactive testing, two recurring patterns appeared. First, the pure LSTM hedges on out-of-distribution positions: when reset to a custom position with no move history, its top-5 spreads roughly uniformly across the few legal moves rather than concentrating on a clear plan. The hybrid, which sees the board, concentrates probability on plausible moves in the same position, though the underlying problem of multi-move planning remains unsolved. Second, the trigram baseline can appear locally confident on tactical recaptures even when the full game position would suggest otherwise — when a piece is hung, the bigram fallback assigns high probability to the recapture because recaptures of just-moved pieces are statistically common, while the LSTM’s full-history view registers the position as unusual and flattens its distribution. The hybrid generally combines both signals: the board confirms the recapture is correct, and the LSTM branch contextualises it within the game flow.

7. Limitations and future work

7.1 No multi-move planning, including basic mates

The most visible limitation is that *none* of the three models reliably finds even simple checkmates. K+Q vs K can be mated by any 1500+ rated human in seconds because the human searches a few moves ahead and recognises the king-corralling pattern. Our models predict one move at a time from learned probability distributions over single-move continuations; they do not search and they do not represent multi-move plans. Consequently they routinely fail to deliver mate-in-1 in positions where a beginner would see it instantly. Search-based reinforcement-learning approaches (MCTS, AlphaZero-style) would address this and are a natural extension; they were out of scope for this project, which is concerned with prediction of human play rather than optimal play.

7.2 Distribution sensitivity

The training corpus is filtered to coherent, skilled play. Predictions degrade on positions far outside that distribution: deliberately constructed odd positions, sequences with multiple deliberate blunders, or starting positions imported from unrelated game formats. Within the distribution (normal Rapid/Classical games at 1500+ Elo) the test numbers are representative; outside it, model behaviour is genuinely uncertain and should not be over-interpreted.

7.3 Single-month, single-Elo-band training data

We trained on Lichess 2017-01 with both sides ≥ 1500 Elo. Cross-month consistency, cross-Elo-band generalisation, and time-of-year drift were not investigated.

7.4 Methodological choices we did not pursue

The project specification explicitly excluded (and we did not implement): Elo-conditioned models in the style of Maia [1], hand-engineered tactical input channels (attack maps, pinned-piece planes, hanging-piece planes), board-mirroring data augmentation, Stockfish-based “best-move” comparison, calibration analysis (ECE, reliability diagrams), and FAISS retrieval against a position database. The most promising open continuation, motivated by Karvonen’s analysis of large language models on chess [2], is grounded natural-language explanation of moves: combining a Maia-style human-prediction layer with a small fine-tuned language model that generates rationale text, validated against a chess engine to suppress hallucinated tactical claims. We plan to pursue this as a TDK research project.

8. Conclusion

Three increasingly informed models trained on the same data and evaluated on the same held-out test split, with monotone improvement at each step:

The n-gram baseline establishes the coverage of pure local pattern matching: top-1 0.252, perplexity 156.7. Its strength is opening memorisation; its weakness is the middlegame. The LSTM adds full-game history and closes the middlegame gap — top-1 0.371, perplexity 10.51. The hybrid CNN+LSTM adds board awareness and consistently improves on the LSTM — top-1 0.408, perplexity 8.03. The hybrid’s improvement is phase-shaped: tied with the LSTM in the opening, growing through the middlegame, and largest in the endgame (top-1 +7.9 pp, perplexity 47 % better). This phase shape is direct evidence that the board signal earns its parameters precisely where sequence context becomes least informative.

The codebase is fully unit-tested (82 tests), seeded, and reproducible from the public Lichess monthly dump. None of the three models reliably finds basic checkmates — predicting *what humans tend to play* is a different problem from *playing optimally*, and search-based methods are required to close that gap.

References

- [1] McIlroy-Young, R., Sen, S., Kleinberg, J., & Anderson, A. (2020). *Aligning Superhuman AI with Human Behavior: Chess as a Model System*. In KDD ‘20: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.
 - [2] Karvonen, A. (2024). *Examining GPT-3.5-turbo-instruct’s Chess Skill*. https://www.adamkarvonen.com/machine_learning/2024/01/03/chess-world-models.html
 - [3] Lichess Open Database. <https://database.lichess.org>
 - [4] Niklasson, F. *python-chess: a chess library for Python*. <https://python-chess.readthedocs.io>
 - [5] Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. In Advances in Neural Information Processing Systems 32.
-

Code and reproducibility: <https://github.com/kornel9/chess-move-prediction>